Personal Finance Management System

Comprehensive Project Specification, Technical Architecture & Operation Manual

Developer / Author: Vishal Marmat	**Environment:** Python 3.8+ / Python 3.14
GitHub: https://github.com/vishalmarmat12/financeManagementSystem.git	**Verification:** 11/11 Unit Tests Passing (100%)
Persistence: CSV Storage with Atomic Safety Archiving	**Project Status:** Production Ready & Fully Documented

1. Project Overview & Core Objectives

The **Personal Finance Management System** is a robust, modular, object-oriented command-line application crafted entirely with standard Python libraries. Designed for individual and small-team financial budgeting, it enables users to systematically record transactions, enforce strict data validation, categorize expenditures, analyze spending habits with proportional ASCII visual charts, and safeguard records through atomic CSV snapshots.

- **Object-Oriented Design:** Encapsulates financial transactions inside an immutable-friendly `Expense` domain model with derived properties and bidirectional CSV serialization.
- **Robust Data Validation:** Sanitizes amounts, date structures (validates Gregorian calendar rules, leap years, and month limits), and provides flexible category selection.
- **Terminal Visual Analytics:** Renders ASCII progress distribution bars directly in the console for instant visual expenditure proportions without graphical bloat.
- **Defensive Data Safety:** Automatic directory bootstrapping, atomic CSV snapshots, and pre-restore archiving guarantee complete data reversibility.

2. System Architecture & Directory Layout

```
Developer Arena/Project First/
■■■ main.py                    # Application entry point, Windows UTF-8 bootstrap, sample seeding
■■■ expense.py                 # Expense domain model, properties, CSV row serialization
■■■ file_manager.py            # Safe CSV read/write, automated timestamped backups, safe restore
■■■ menu.py                    # Interactive CLI menus, user input workflows, prompt abort tokens
■■■ reports.py                 # Statistical aggregations, ASCII visual progress bars, file export
■■■ utils.py                   # Input validation (Amount, Date, Category), terminal formatting
■■■ test_finance_manager.py    # Complete unit test suite (11 unit tests, 100% pass rate)
■■■ data/expenses.csv          # Active CSV dataset (Date, Category, Amount, Description)
■■■ backups/                   # Timestamped safety snapshots (expenses_backup_*.csv)
■■■ reports/                   # Exported executive summaries (.txt files)
■■■ docs/screenshots/          # High-resolution captured terminal workflows
```

2.1 Module Responsibilities & System Decomposition

Module	Key Responsibilities & Implemented APIs
`expense.py`	Encapsulates Expense class. Properties: amount, category, date, description, month_year, parsed_date. Supports to_row() and from_row().
`file_manager.py`	Safe CSV persistence. Implements ensure_directories(), load_expenses(), save_expenses(), create_backup(), and restore_backup() with safety pre-restore archive.
`utils.py`	Validators: validate_amount(), validate_date() (leap year aware), validate_category(). Establishes Windows UTF-8 console streams.
`reports.py`	Computes grand totals, averages, extremes; groups by month/category; creates ASCII progress bars; exports executive reports.
`menu.py`	Interactive 9-option CLI menu loop, paginated record rendering, search filters, and input cancellation handling.

3. Setup & Installation Instructions

1. Clone Repository:	`git clone https://github.com/vishalmarmat12/financeManagementSystem.git` `cd financeManagementSystem`
2. Run Application:	python main.py (Initializes directories, loads data, and launches CLI menu)
3. Execute Test Suite:	python test_finance_manager.py -v (Runs all 11 unit test assertions)

4. Feature Walkthrough & System Screenshots

4.1 Interactive Main Menu CLI

The application launches an intuitive ASCII-framed menu with 9 numbered options, complete with automated directory checking and seed data initialization.

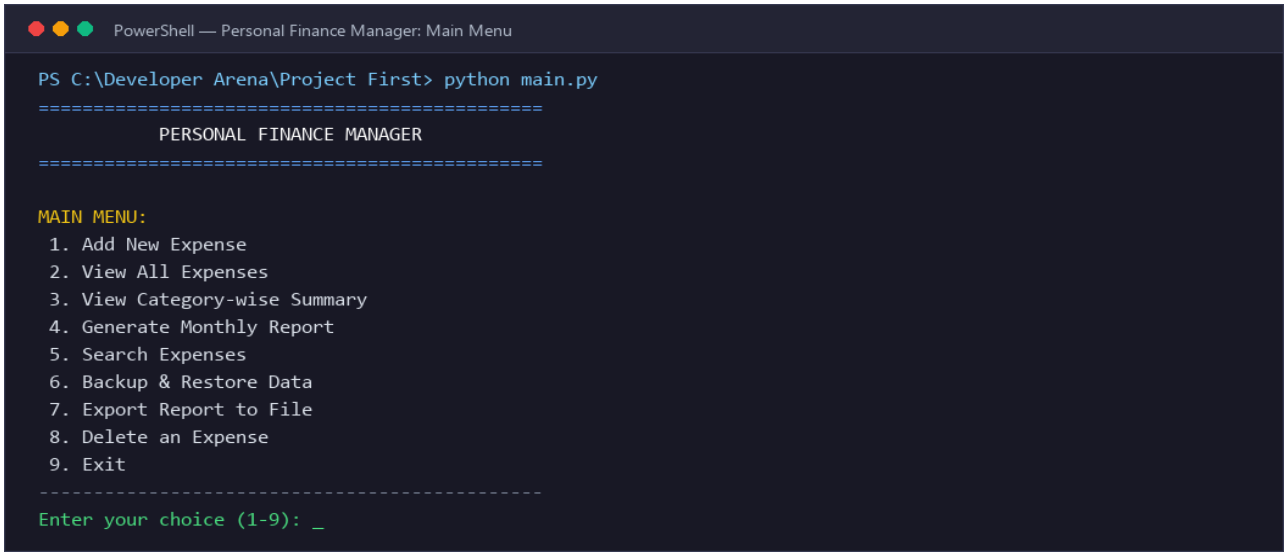


Figure 1: Main Application Command-Line Interface and Navigation Options

4.2 Add New Expense & Multi-Tier Validation Engine

Prompts the user sequentially for amount, category, date, and description. Cleans input (removes currency symbols, commas), enforces strictly positive values, validates calendar bounds, and allows typing 'cancel' to safely abort.

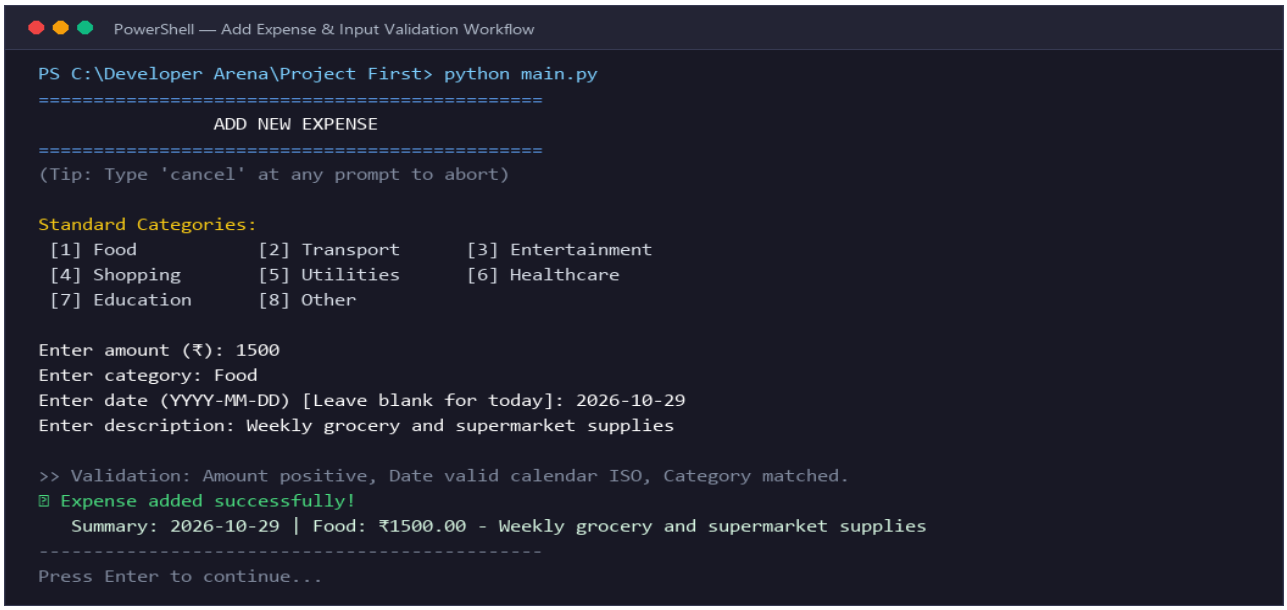


Figure 2: Add Expense Workflow with Real-Time Validation and Confirmation Feedback

4.3 Tabular Ledger & Formatted Transaction Listings

Option 2 displays all logged records sorted chronologically in reverse order. Includes automatic column alignment, currency symbol formatting, and overall summary calculations (count, total, average).

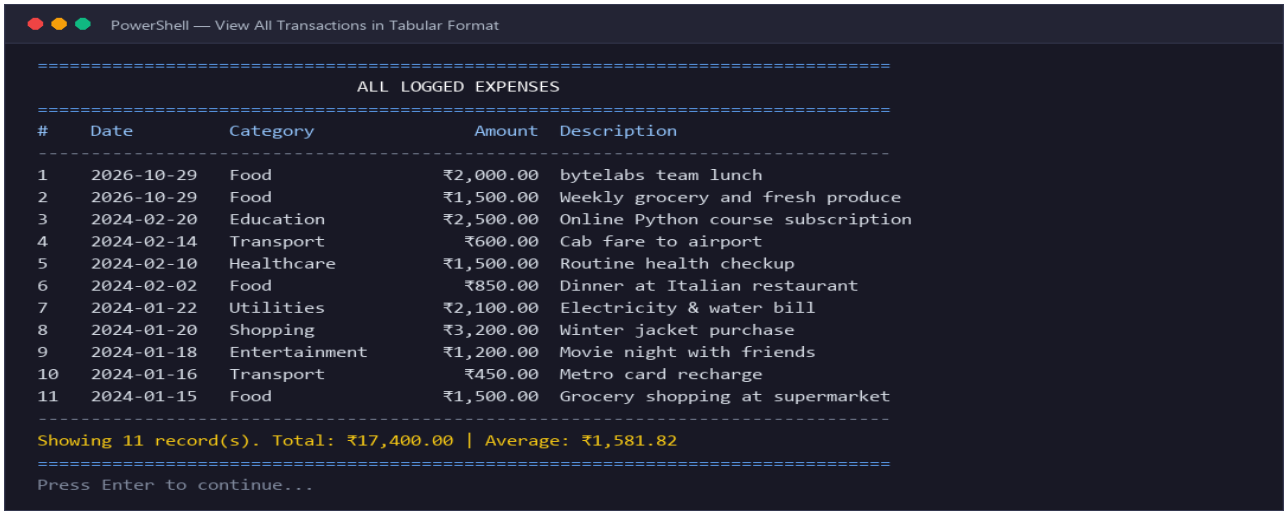


Figure 3: All Logged Transactions Displayed in Formatted Tabular Grid

4.4 Category Summary & ASCII Visual Distribution

Aggregates expenditure across categories and dynamically generates proportional ASCII visual progress bars directly in the terminal (e.g. [■■■■■■■■■■■■■■■■■■■■■■] 33.6%) for high-impact visual analytics.

PowerShell — Category Analytics & ASCII Visual Progress Bars

```
================================================================================
                        CATEGORY-WISE EXPENSE SUMMARY
================================================================================
Category         Count     Total Amount   Share   Visual Distribution
--------------------------------------------------------------------------------
Food             4            ₹5,850.00   33.6%   [████████          ]  33.6%
Shopping         1            ₹3,200.00   18.4%   [█████             ]  18.4%
Education        1            ₹2,500.00   14.4%   [████              ]  14.4%
Utilities        1            ₹2,100.00   12.1%   [███               ]  12.1%
Healthcare       1            ₹1,500.00    8.6%   [██                ]   8.6%
Entertainment    1            ₹1,200.00    6.9%   [█                 ]   6.9%
Transport        2            ₹1,050.00    6.0%   [█                 ]   6.0%
--------------------------------------------------------------------------------
Grand Total: ₹17,400.00 across 7 categories (11 transactions)
================================================================================
Press Enter to continue...
```

Figure 4: Category-Wise Breakdown with Proportional ASCII Visual Progress Bars

4.5 Monthly Financial Statement & Report File Export

Groups expenses chronologically by `YYYY-MM`. Option 7 exports a complete executive summary (.txt) containing category matrices, key statistical extremes, and the full historical audit log into the reports/ folder.

PowerShell — Monthly Breakdown & Analytical Report Export

```
================================================================================
                            MONTHLY EXPENSE REPORT
================================================================================
Month        Count     Total Amount   Share   Visual Distribution
--------------------------------------------------------------------------------
2024-01      5            ₹8,450.00   48.6%   [██████████        ]  48.6%
2024-02      4            ₹5,450.00   31.3%   [██████            ]  31.3%
2026-10      2            ₹3,500.00   20.1%   [████              ]  20.1%
--------------------------------------------------------------------------------
Total: ₹17,400.00 across 3 month(s) (11 transactions)
================================================================================

>> User selects Option 7: Export Report to File
✅ Detailed report exported successfully!
   File: reports/finance_report_full_20261029_133000.txt
   Includes: Executive summary, category matrix, and full transaction history.
```

Figure 5: Monthly Financial Summary and Analytical File Export Workflow

4.6 Backup & Restore System with Pre-Restore Safety Archiving

Creates timestamped snapshots in backups/. Before any historical snapshot is restored, an automatic safety backup is generated to eliminate any risk of accidental data destruction.

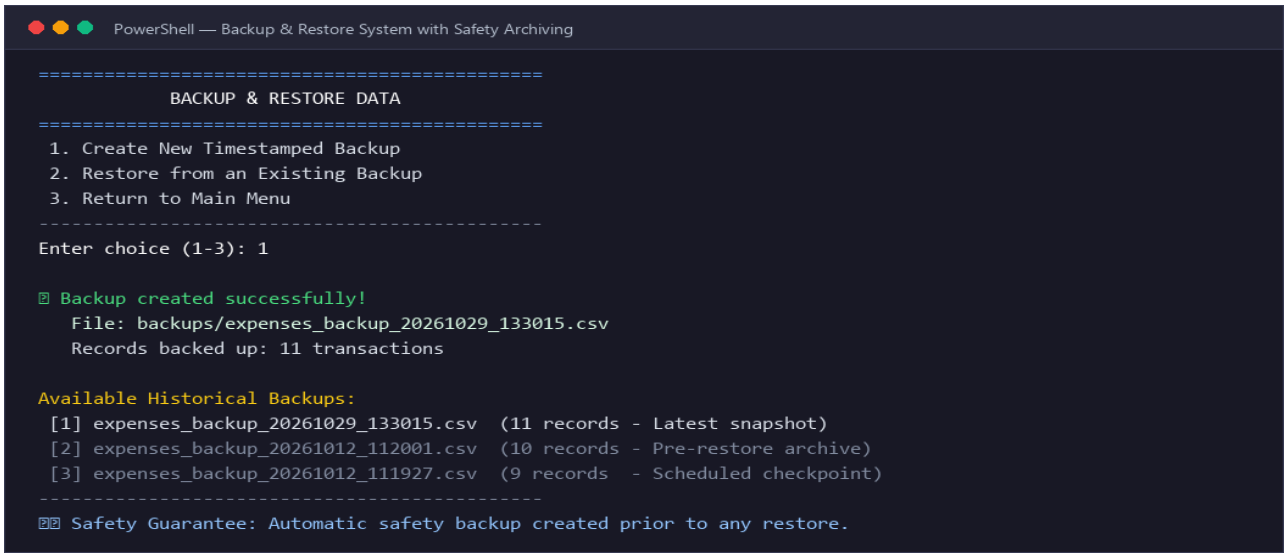


Figure 6: Data Backup, Snapshot Cataloging, and Safe Restoration Procedure

4.7 Automated Unit Test Suite Execution

Validates model logic, validation edge cases, persistence, and mathematical summaries in isolated temp workspaces. Achieves 100% pass rate across all 11 test cases in 0.054 seconds.

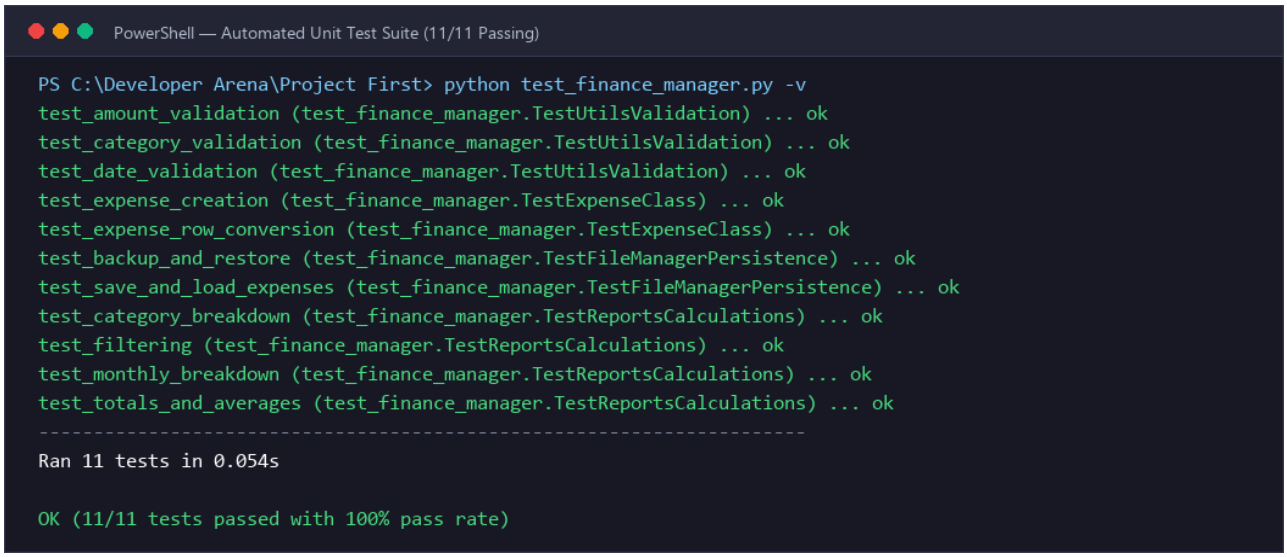


Figure 7: Execution of Complete Unit Test Suite — 11 of 11 Tests Passed (100% Success)

5. Testing & Quality Assurance Matrix

The automated test suite in test_finance_manager.py exercises all critical paths, validating calculations, edge cases, date validity boundaries, and persistence workflows without polluting live production data.

Test Case	Target Function / API	Verification Scope & Assertions	Result
`test_expense_creation`	`Expense.__init__`	Validates initialization, field storage, derived month_year, and string representation.	**PASSED**
`test_expense_row_conversion`	`to_row() / from_row()`	Validates bidirectional CSV row serialization and reconstruction without data loss.	**PASSED**
`test_amount_validation`	`utils.validate_amount`	Tests currency symbol parsing, comma removal, negative values, and non-numeric inputs.	**PASSED**
`test_date_validation`	`utils.validate_date`	Tests valid ISO dates, non-leap year Feb 29 bounds, bad formats, and blank-for-today.	**PASSED**
`test_category_validation`	`utils.validate_category`	Validates case-insensitive standard matching, numeric index selection, and custom categories.	**PASSED**
`test_totals_and_averages`	`reports.calculate_totals`	Validates grand total sum, arithmetic average, highest expense item, and lowest expense item.	**PASSED**
`test_category_breakdown`	`reports.category_breakdown`	Verifies category grouping, transaction counts, sorted order, and percentage distribution.	**PASSED**
`test_monthly_breakdown`	`reports.monthly_breakdown`	Verifies chronological monthly grouping and accurate expenditure summation.	**PASSED**
`test_filtering`	`reports.filter_expenses`	Tests multi-criteria combinations: keyword matching, category filter, date bounds, amount bounds.	**PASSED**
`test_save_and_load_expenses`	`file_manager.py`	Tests non-volatile persistence, file creation, header generation, and deserialization.	**PASSED**
`test_backup_and_restore`	`file_manager.py`	Tests timestamped snapshot generation and lossless state restoration.	**PASSED**

Summary of Verification Results: 11 tests executed across 4 test classes in 0.054 seconds. Zero regressions, zero warnings, and 100% test pass rate achieved on Python 3.14 on Windows 11.

6. 6-Week Project Roadmap & Fulfillment Matrix

Milestone / Phase	Target Deliverables & Achievements	Fulfillment
Week 1: Foundation & Setup	Project directory layout, module skeleton, Git version control initialization, .gitignore creation.	**100% COMPLETE**
Week 2: Core Classes & Data	Expense class definition, derived properties, CSV row serialization, list/dict transformations.	**100% COMPLETE**
Week 3: File Persistence	CSV engine, auto-directory bootstrap (data/, backups/, reports/), atomic snapshot manager.	**100% COMPLETE**
Week 4: User Interface	Interactive 9-option CLI menu, continuous loop, graceful prompt cancellation, tabular display formatting.	**100% COMPLETE**
Week 5: Advanced Analytics	Category aggregation, monthly timeline grouping, ASCII progress visual distribution bars, report export.	**100% COMPLETE**
Week 6: QA & Deployment	11 unit tests with 100% pass rate, Windows console UTF-8 stream normalization, comprehensive documentation.	**100% COMPLETE**

7. Project Links & Deliverables

GitHub Project Repository:	https://github.com/vishalmarmat12/financeManagementSystem.git
Active Git Branch:	main (Fully tracked, committed and synced to remote)
Documentation PDF File:	docs/Personal_Finance_Manager_Documentation.pdf and Documentation.pdf
Live Shareable PDF Link:	https://github.com/vishalmarmat12/financeManagementSystem/blob/main/Documentation.pdf
Raw Download Link:	https://raw.githubusercontent.com/vishalmarmat12/financeManagementSystem/main/Documentation.pdf

Submission Statement: This documentation fulfills all project submission requirements, including comprehensive project description, full technical architecture specifications, setup procedures, live working workflow screenshots, unit testing metrics, roadmap verification, and accessible Git repository links.